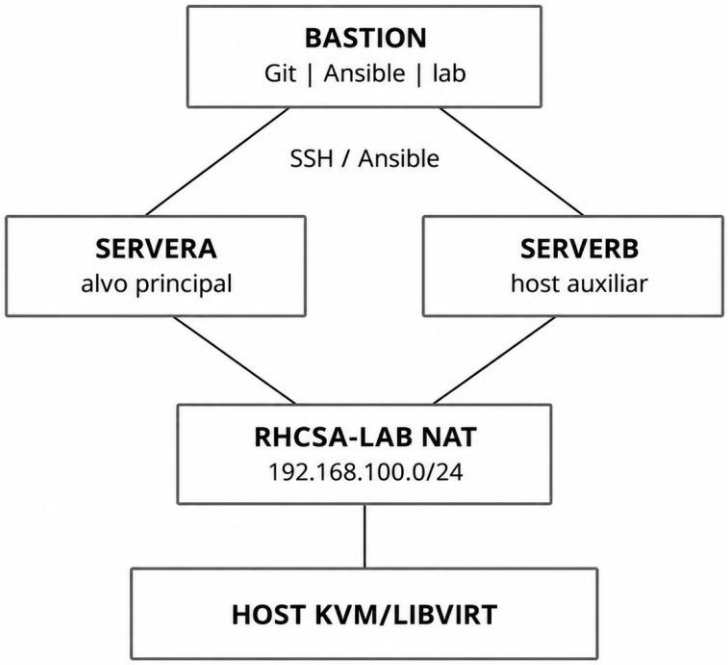
RHCSA EX200 v10 LAB

MANUAL DE IMPLANTAÇÃO, OPERAÇÃO E ESTUDOS

Ambiente hands-on independente para administração Red Hat Enterprise Linux
com foco no RHCSA / EX200



Eduardo Andrade

Edição 1.1 | Base funcional v1.8.4 | Outubro de 2026

Projeto independente • Apache License 2.0

Apresentação

Este manual consolida a arquitetura, a implantação, a operação, a metodologia de estudos, o catálogo de laboratórios, os mecanismos de segurança e a engenharia do projeto RHCSA EX200 v10 Lab. O conteúdo foi organizado a partir do repositório fornecido para esta edição, preservando os procedimentos operacionais e comandos necessários para que outra pessoa consiga reproduzir o ambiente.

O projeto é independente. Ele não é afiliado, patrocinado ou publicado pela Red Hat e não contém questões reais de certificação. Os exercícios são cenários autorais voltados à prática das competências de administração Linux/RHEL.

> ***Observação:*** *O termo "v10" no nome do projeto indica o alvo de estudo baseado em RHEL 10 / EX200. A plataforma local atualmente validada pelo projeto é RHEL 9.8.*

Como utilizar esta apostila

- Para uma instalação nova, siga a Parte II na ordem apresentada.
- Para uso diário, consulte a Parte III e o capítulo do CLI lab.
- Para estudar por domínio, utilize o catálogo de 100 itens da Parte IV.
- Para manutenção do projeto, validação e releases, utilize a Parte V.
- Os procedimentos destrutivos de armazenamento devem respeitar estritamente o contrato de segurança apresentado neste manual.

Sumário

Apresentação 2
Como utilizar esta apostila 2
1. Visão geral do projeto 6
2. Arquitetura, rede e armazenamento 7
2.1 Endereçamento 7
2.2 Topologia de discos 8
3. Metodologia, compatibilidade e currículo 9
3.1 Quatro princípios do laboratório 9
3.2 Compatibilidade 9
3.3 Distribuição do currículo 9
4. Roteiro geral de instalação 11
5. Requisitos e arquitetura 12
Plataforma e requisitos 12
Plataforma e requisitos 12
Imagens e downloads oficiais 12
Configuração real do hospedeiro 13
Rede isolada de prática 14
Critério para avançar 15
6. Preparação do host KVM/libvirt 16
6.1 Caminho A — host Ubuntu 24.04 / 26.04 16
6.2 Caminho B — host RHEL 9 / Rocky Linux 9 / AlmaLinux 9 16
6.3 Validação comum depois do caminho escolhido 17
6.4 Preparar o pool de storage 17
6.5 Criar a rede NAT rhcsa-lab 17
6.6 Validar a imagem-base 18
6.7 Preservar a imagem-mestre 18
7. Criação das máquinas virtuais 19
7.1 Criar os discos das VMs 19
7.2 Criar o bastion 19
7.3 Criar o servera 19
7.4 Criar o serverb 20
7.5 Validar as VMs 20
7.6 Abrir os consoles 20
8. Configuração inicial das VMs 21
8.1 Bastion 21
8.2 servera 22
8.3 serverb 24

8.4 Critério para avançar ... 26
9. Preparação do bastion, Git e SSH ... 27
9.1 Validar a rede a partir do bastion ... 27
9.2 Criar a chave SSH dedicada ... 27
9.3 Conferir fingerprints dos hosts ... 27
9.4 Copiar a chave para os nós ... 27
9.5 Criar ~/.ssh/config e validar sudo não interativo ... 27
9.6 Clonar o projeto somente depois de rede, SSH e sudo estarem prontos ... 28
9.7 Instalar o CLI lab ... 28
10. Configuração e validação do Ansible ... 29
10.1 Validar Ansible e inventário ... 29
10.2 Como o bootstrap corrigido funciona ... 29
10.3 Instalar e validar dependências ... 29
10.4 Executar as validações explícitas ... 30
10.5 Validar o CLI e o catálogo ... 30
11. Baseline e snapshots ... 31
11.1 Verificar discos e snapshots existentes ... 31
11.2 Desligar os nós gerenciados ... 31
11.3 Criar baseline-pre-labs ... 31
11.4 Validar snapshots e reiniciar ... 31
12. Primeiro exercício ... 32
12.1 Check final do ambiente ... 32
12.2 Começar a estudar ... 32
12.3 Checklist final ... 32
12.4 Atalho operacional para os próximos dias ... 32
12.5 Regras operacionais ... 33
13. CLI lab e fluxo operacional ... 34
13.1 Fluxo de uma sessão ... 34
14. Estratégia de estudo e avaliação por estado final ... 35
14.1 Rotina recomendada ... 35
15. Catálogo completo dos 100 itens ... 37
Objective 01 — Essential Tools ... 37
Objective 02 — Users & Groups ... 37
Objective 03 — RPM, DNF & Repositories ... 38
Objective 04 — Simple Shell Scripts ... 38
Objective 05 — Processes, Scheduling, Tuning & systemd ... 38
Objective 06 — NetworkManager, IP Networking & Name Resolution ... 39
Objective 07 — Scheduling, Temporary Files, Logging & Time ... 39
Objective 08 — Partitions, Filesystems, Swap & LVM ... 40

Objective 09 — Boot, GRUB & Recovery 40
Objective 10 — SELinux & firewalld 41
Objective 11 — NFS & AutoFS 41
Objective 12 — Comprehensive EX200 Challenges 42
15.1 Totais operacionais 42
16. Boot, recuperação manual e segurança de armazenamento 43
16.1 Dois exercícios deliberadamente manuais 43
16.2 Contrato de segurança do Objective 08 43
17. Estrutura do repositório e contrato de um lab 45
17.1 Contrato de um laboratório ready 45
17.2 Separação de responsabilidades 45
18. Release gate, integração de referência e manutenção 46
18.1 Gate estático 46
18.2 Integração de referência 46
18.3 Checklist de release 46
18.4 Fluxo Git recomendado para manutenção documental 46
19. Releases e roadmap 47
19.1 Estado do roadmap 47
20. Referência rápida 50
20.1 Comandos do estudante 50
20.2 Comandos de diagnóstico do ambiente 50
20.3 Valores esperados de rede 50
20.4 Downloads e repositório 50
21. Autoria, apoio, licença e marcas 51
21.1 Autor e perfis profissionais 51
21.2 Apoie o projeto 51
21.3 Independência e marcas 51
21.4 Licença 51
Apêndice A — Arquivo LICENSE (Apache License 2.0) 52

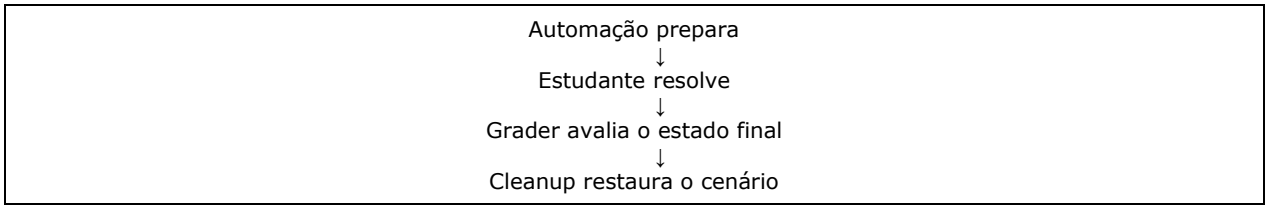
PARTE I

Visão geral, arquitetura e princípios

RHCSA EX200 v10 Lab

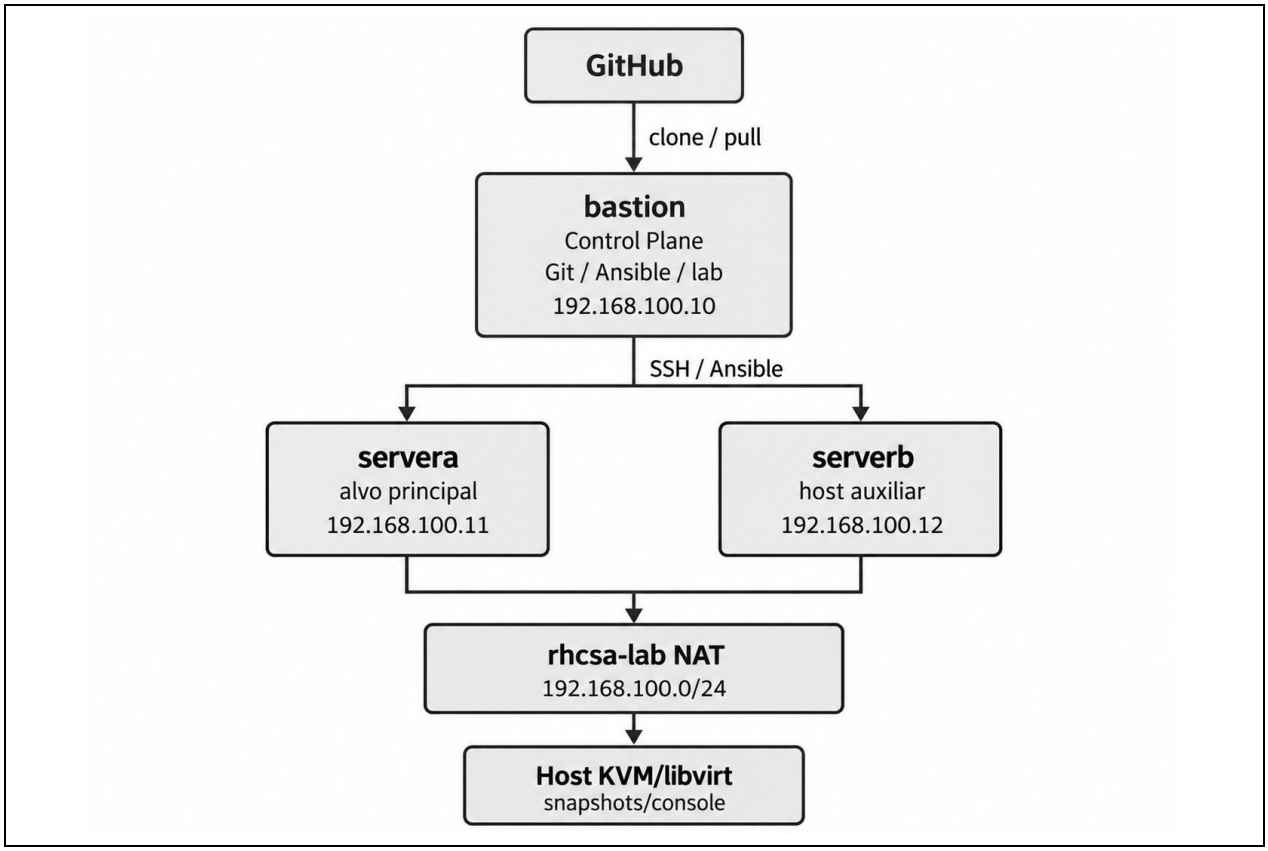
1. Visão geral do projeto

O RHCSA EX200 v10 Lab é um ambiente local, repetível e orientado a estado final para prática de administração Linux/RHEL. O laboratório utiliza três máquinas virtuais: `bastion`, `servera` e `serverb`. A automação prepara o cenário, valida dependências, executa graders e restaura o ambiente; a resolução das tarefas permanece manual.



Item	**Estado**
Currículo	Concluído
Objetivos	12
Itens de aprendizagem	100
Labs automatizados e avaliáveis em runtime	98
Drills manuais por console	2
CLI lab	Implementado
Setup / grade / cleanup	Implementados
Release gate	Implementado
Regression runner de referência	Implementado
Plataforma local	RHEL 9.8
Alvo de estudo	RHEL 10 / RHCSA EX200
Virtualização	KVM/libvirt
Licença	Apache License 2.0
Base estável	v1.8.4

2. Arquitetura, rede e armazenamento



Componente	Responsabilidade
bastion	Git, Ansible, CLI lab, graders, estado dos exercícios e automação
servera	Principal máquina de prática do estudante
serverb	NFS, AutoFS, SSH, rede, serviços auxiliares e cenários distribuídos
Host KVM	Criação das VMs, discos, console, snapshots e recuperação out-of-band

2.1 Endereçamento

Nó	IPv4	Papel
bastion	192.168.100.10	Plano de controle
servera	192.168.100.11	Alvo principal
serverb	192.168.100.12	Host auxiliar
Gateway libvirt	192.168.100.1	NAT / saída da rede
Domínio local	lab.test	Resolução local

2.2 Topologia de discos

VM	Disco	Tamanho	Uso
bastion	vda	30 GiB	Sistema operacional
servera	vda	30 GiB	Sistema operacional — protegido
servera	vdb	5 GiB	Scratch
servera	vdc	5 GiB	Scratch
servera	vdd	3 GiB	Scratch
servera	vde	2 GiB	Scratch
serverb	vda	30 GiB	Sistema operacional — protegido
serverb	vdb	5 GiB	Scratch
serverb	vdc	3 GiB	Scratch

3. Metodologia, compatibilidade e currículo

3.1 Quatro princípios do laboratório

- O estudante resolve manualmente: Ansible prepara e restaura o ambiente, mas não executa a tarefa do aluno.
- O grader avalia o estado final: uma solução válida não depende de sequência única de comandos.
- O cleanup faz parte do contrato: `lab finish` deve restaurar o cenário previsto.
- O disco do sistema é protegido: exercícios destrutivos de storage são restritos aos discos scratch autorizados.

3.2 Compatibilidade

Nível	Significado
exact	Execução equivalente no ambiente local e no alvo de estudo
concept-and-state	Conceito e estado final alinhados; podem existir diferenças operacionais entre versões
manual-console-rhel10-aligned	Prática manual por console alinhada ao alvo RHEL 10

Observação: *A matriz efetiva dos 100 itens nesta edição contém 52 labs exact, 46 concept-and-state e 2 exercícios manual-console-rhel10-aligned.*

3.3 Distribuição do currículo

Objective	Tema	Itens
01	Essential Tools	10
02	Users & Groups	8
03	RPM, DNF & Repositories	8
04	Simple Shell Scripts	8
05	Processes, Scheduling, Tuning & systemd	10
06	Networking / NetworkManager	8
07	Scheduling, Temporary Files, Logging & Time	8
08	Partitions, Filesystems, Swap & LVM	12
09	Boot, GRUB & Recovery	7
10	SELinux & firewalld	9
11	NFS & AutoFS	6
12	Comprehensive EX200 Challenges	6
Total		100

PARTE II

Implantação completa do laboratório

RHCSA EX200 v10 Lab

4. Roteiro geral de instalação

A implantação segue oito etapas. Os capítulos seguintes incorporam o conteúdo operacional completo da documentação `docs/setup/`, consolidado em formato de apostila.

Etapa	Conteúdo
01	Requisitos e arquitetura
02	Preparação do host KVM/libvirt
03	Criação das VMs — Método A ou Método B
04	Configuração inicial das VMs
05	Bastion, Git e SSH
06	Ansible e validação
07	Baseline e snapshots
08	Primeiro exercício

> ***Observação:*** *Os procedimentos do Método A e do Método B convergem para a mesma topologia final. Depois da criação das VMs, a configuração segue o mesmo caminho.*

5. Requisitos e arquitetura

Índice · Home · Próximo

Este roteiro começa com um host Linux e termina com o primeiro exercício executado pelo estudante.

A construção das máquinas virtuais pode ser realizada por dois métodos:

- **Método A — instalação manual:** criação das VMs e instalação do RHEL a partir da mídia ISO oficial;
- **Método B — imagem-base:** implantação rápida a partir de uma imagem QCOW2 previamente preparada.

Os dois métodos convergem para a mesma arquitetura final. O Ansible entra posteriormente, no `bastion`, para preparar, restaurar e avaliar os cenários dos laboratórios.

Plataforma e requisitos

Plataforma e requisitos

A base atual do projeto utiliza **RHEL 9.8 nas três VMs** e toma **RHEL 10 como referência de estudo e classificação das competências**.

O playbook `ansible/playbooks/validate.yml` da versão atual valida a base RHEL utilizada pelo projeto. A adoção de RHEL 10 como sistema operacional local exige revisão prévia dessa validação e da compatibilidade dos exercícios; não é suficiente apenas substituir a mídia de instalação.

O hospedeiro utilizado originalmente no desenvolvimento do projeto é **Ubuntu Desktop 24.04.5 LTS (Noble Numbat)** com KVM/libvirt. O guia também contempla sistemas da família RHEL como hospedeiros KVM, mantendo a distribuição do host independente da distribuição instalada nas VMs.

Independentemente do hospedeiro escolhido, a arquitetura lógica do laboratório permanece a mesma.

Imagens e downloads oficiais

Uso	Distribuição / imagem	Link oficial
Host Ubuntu	Ubuntu Desktop 24.04 LTS, ISO AMD64	Imagens da série 24.04
Host alternativo	Rocky Linux 9.8 x86_64, DVD com ambiente gráfico	Diretório oficial de ISOs 9.8
Host alternativo, mídia completa	Rocky Linux 9.8 DVD x86_64	Baixar ISO DVD
VMs da base atual	RHEL 9.8, Binary DVD x86_64	Red Hat Developer · Portal de downloads RHEL

Escolha da origem das VMs

A mídia necessária depende do método escolhido na etapa 03.

Método	Origem	Necessidade
Método A	ISO Binary DVD x86_64 do RHEL 9.8	Obrigatória
Método B	Imagem-base rhel-rhcsa-base-v1.qcow2	Obrigatória
Método B	Arquivo rhel-rhcsa-base-v1.qcow2.sha256	Recomendado para validação de integridade

No **Método A**, cada VM é criada e instalada individualmente a partir da mídia oficial do RHEL.

No **Método B**, a imagem-base é utilizada somente como origem para a criação de discos independentes de `bastion`, `servera` e `serverb`. A imagem original não deve ser utilizada diretamente como disco ativo de uma VM.

O repositório Git não armazena ISOs nem imagens QCOW2 do Red Hat Enterprise Linux.

O padrão deste laboratório é **hospedeiro Desktop, com interface gráfica**. Use Ubuntu Desktop; na alternativa Rocky Linux, instale pela mídia DVD e selecione um ambiente com interface gráfica no instalador. Isso permite usar navegador, virt-manager e consoles localmente. Esse padrão é do hospedeiro; as VMs RHEL continuam com instalação mínima conforme a etapa 3. Use a ISO da série 24.04 disponível na página oficial, sem confundir a atualização da mídia com a versão exata do host original.

O download RHEL depende do acesso autorizado à conta Red Hat. No portal, selecione RHEL 9.8 e a mídia **Binary DVD x86_64** para reproduzir a base. Links diretos autenticados/temporários da ISO não são publicados no projeto. A ISO Boot depende de uma fonte de pacotes durante a instalação; ela não substitui a DVD no procedimento de repositório local.

Confira checksums nas páginas oficiais: SHA256SUMS do Ubuntu e CHECKSUM do Rocky 9.8. Compare o arquivo exato baixado e, para conferir a origem, valide também a assinatura com a chave oficial da distribuição.

Rocky como host ou como VM?

Como hospedeiro: pode executar KVM/libvirt com as VMs RHEL do projeto; siga o caminho Rocky na etapa 2. Essa alternativa está documentada, mas não foi validada neste projeto durante a geração do pacote.

Como sistema das VMs: Rocky Linux 9.8 é uma opção para adaptar os exercícios gerais de Linux, mas não é uma substituição automática no projeto atual. O playbook `ansible/playbooks/validate.yml` exige `ansible_distribution == "RedHat"`; com Rocky ele falhará. Também é necessário revisar repositórios, chaves GPG, pacotes e classificações de compatibilidade antes de considerar essa variante validada. Não use a configuração de repositórios RHEL/DVD como se fosse Rocky. A base de referência continua RHEL 9.8 nas VMs.

Configuração real do hospedeiro

Item	Registro disponível
Equipamento	ThinkPad T430 (identificação informada no terminal do autor)
Sistema	Ubuntu Desktop 24.04.5 LTS (Noble Numbat), x86_64
Virtualização	KVM/QEMU gerenciado por libvirt
Processador	Intel Core i5-3320M, frequência nominal 2,60 GHz; máxima reportada 3,30 GHz
Topologia de CPU	1 socket, 2 núcleos físicos, 2 threads por núcleo: 4 CPUs lógicas
Virtualização da CPU	Intel VT-x (vmx no resultado de lscpu)
Memória RAM	16 GiB reportados por free -h
Swap	4,0 GiB
Disco físico	SSD Kingston SA400S37240G, capacidade comercial 240 GB; 223,6 GiB exibidos por lsblk
Tipo de armazenamento	Não rotacional (ROTA=0 no SSD)
Rede das VMs	192.168.100.0/24; gateway 192.168.100.1
Discos virtuais dos alvos	Sistema 30 GiB; discos extras conforme tabela abaixo

Configuração registrada a partir da coleta fornecida pelo autor em 02/10/2026. Os 4 processadores lógicos correspondem a **2 núcleos físicos com Hyper-Threading**, não a 4 núcleos físicos. A coleta identifica VT-x; a validação operacional de KVM continua na etapa 2.

Os valores momentâneos de memória usada/disponível não são requisitos do projeto e não foram tratados como capacidade fixa. A coleta não informa o espaço livre do filesystem nem confirma o diretório físico dos discos das VMs. Antes de uma nova instalação, confira `df -h /var/lib/libvirt/images` (ou o caminho real do pool).

A tabela de recursos abaixo é **dimensionamento sugerido para uma nova instalação**, não a alocação medida das VMs atuais. As três VMs propostas somam 7 GiB de RAM e 6 vCPUs; vCPUs são uma alocação compartilhada sobre as 4 CPUs lógicas do host. O desempenho depende da carga simultânea e dos recursos consumidos pelo Ubuntu.

Para conferir a configuração de outro hospedeiro, execute nele:

```
cat /etc/os-release
lscpu
free -h
lsblk -d -o NAME,SIZE,MODEL,ROTA
sudo virsh -c qemu:///system net-list --all
sudo virsh -c qemu:///system pool-list --all
```

Não execute a coleta dentro de bastion: ela mostraria os recursos da VM. `ROTA=0` normalmente indica armazenamento não rotacional; `ROTA=1` indica rotacional conforme exposto pelo dispositivo.

Dimensionamento inicial sugerido, não um mínimo certificado:

Máquina	vCPU	RAM	Disco do sistema	Discos extras
bastion	2	2 GiB	30 GiB	nenhum
servera	2	3 GiB	30 GiB	5 + 5 + 3 + 2 GiB
serverb	2	2 GiB	30 GiB	5 + 3 GiB

Reserve memória para o host; 16 GiB de RAM é uma configuração inicial confortável. Reserve pelo menos 120 GiB livres para sistemas, ISO e crescimento dos snapshots. A necessidade real varia com exercícios e retenção. SSD é preferível.

Pré-requisitos para iniciar a instalação

Independentemente do método escolhido:

- CPU x86_64 com AMD-V ou Intel VT-x habilitado no firmware;
- host Linux com KVM/QEMU e libvirt;
- acesso administrativo ao hospedeiro;
- espaço em disco suficiente para as VMs, imagens e snapshots;
- conexão para obtenção do projeto e das dependências;
- faixa de rede disponível para o laboratório.

Para o **Método A**, também é necessária uma mídia ISO DVD do RHEL obtida por meio de acesso autorizado à Red Hat.

Para o **Método B**, é necessária uma imagem-base QCOW2 compatível com a versão documentada pelo projeto e, preferencialmente, seu respectivo arquivo de checksum SHA-256.

ISOs, credenciais Red Hat e outros materiais proprietários não são armazenados no repositório Git.

Rede isolada de prática

Elemento	Endereço	Papel
Rede libvirt rhcsa-lab	192.168.100.0/24	NAT do laboratório

Elemento	Endereço	Papel
Interface do host virbr-rhcsa	192.168.100.1	gateway/DNS das VMs
bastion	192.168.100.10	estação do aluno e automação
servera	192.168.100.11	alvo principal
serverb	192.168.100.12	alvo auxiliar

Domínio usado no roteiro: lab.test. Verifique se 192.168.100.0/24 conflita com LAN, VPN ou outra rede libvirt. Se conflitar, escolha outra faixa e atualize rede, VMs, resolução de nomes e inventário juntos.

O host KVM executa virsh e cria as VMs. O bastion executa Git, Ansible e lab. As tarefas dos exercícios são resolvidas manualmente em servera/serverb. Não é necessário clonar o projeto no host para seguir as etapas 2 a 4.

Critério para avançar

Antes de seguir para a preparação do host KVM/libvirt, confirme:

```
[ ] virtualização por hardware disponível
[ ] KVM/libvirt poderá ser utilizado no hospedeiro
[ ] memória e armazenamento suficientes para as três VMs
[ ] faixa 192.168.100.0/24 sem conflito com LAN, VPN ou outra rede libvirt
[ ] Método A ou Método B escolhido
[ ] origem necessária para o método escolhido disponível

[Índice](README.md) · [Home](../../README.md) · [Próximo](02-host-kvm.md)
```

6. Preparação do host KVM/libvirt

Este capítulo reproduz o caminho operacional validado pelo Runbook do projeto. A partir daqui, os comandos foram padronizados para que o manual e o Runbook utilizem a mesma sequência.

> *Escolha apenas um caminho de preparação do host: Ubuntu ou família RHEL. Depois disso, os procedimentos convergem para o mesmo pool, a mesma rede libvirt e a mesma imagem-base.*

6.1 Caminho A — host Ubuntu 24.04 / 26.04

Execute no host Ubuntu:

```
hostnamectl
lscpu
free -h
lsblk

sudo apt update

sudo apt install -y \
  qemu-kvm \
  qemu-utils \
  libvirt-daemon-system \
  libvirt-clients \
  virtinst \
  virt-manager \
  virt-viewer \
  cpu-checker \
  libosinfo-bin \
  cockpit \
  cockpit-machines

sudo systemctl enable --now libvirtd
sudo systemctl enable --now cockpit.socket

sudo usermod -aG kvm,libvirt "$USER"

kvm-ok
sudo virt-host-validate qemu
sudo virsh -c qemu:///system list --all
```

6.2 Caminho B — host RHEL 9 / Rocky Linux 9 / AlmaLinux 9

Execute no host da família RHEL:

```
hostnamectl
lscpu
free -h
lsblk

sudo dnf makecache
sudo dnf update -y

sudo dnf install -y \
  qemu-kvm \
  qemu-img \
  libvirt \
  libvirt-client \
  virt-install \
  virt-manager \
  virt-viewer \
  libosinfo \
  osinfo-db-tools \
  cockpit \
  cockpit-machines

sudo systemctl enable --now libvirtd
sudo systemctl enable --now cockpit.socket

sudo usermod -aG kvm,libvirt "$USER"

egrep -c '(vmx|svm)' /proc/cpuinfo
```

```
lsmod | grep kvm

sudo virt-host-validate qemu
sudo virsh -c qemu:///system list --all
```

6.3 Validação comum depois do caminho escolhido

Faça logout/login para que a associação aos grupos kvm/libvirt seja aplicada. Depois valide:

```
id

sudo virsh -c qemu:///system list --all
sudo virsh -c qemu:///system net-list --all
sudo virsh -c qemu:///system pool-list --all

systemctl is-active cockpit.socket
```

6.4 Preparar o pool de storage

O Runbook foi ajustado para reutilizar o pool default quando ele já existe e criá-lo somente quando necessário.

```
sudo virsh -c qemu:///system pool-list --all

sudo virsh -c qemu:///system pool-start default 2>/dev/null || true
sudo virsh -c qemu:///system pool-autostart default 2>/dev/null || true

sudo mkdir -p /var/lib/libvirt/images

if ! sudo virsh -c qemu:///system pool-info default >/dev/null 2>&1; then
  sudo virsh -c qemu:///system \
    pool-define-as default dir \
    --target /var/lib/libvirt/images

  sudo virsh -c qemu:///system pool-start default
  sudo virsh -c qemu:///system pool-autostart default
fi

sudo virsh -c qemu:///system pool-info default
```

6.5 Criar a rede NAT rhcsa-lab

Antes de criar a rede, confira se 192.168.100.0/24 não conflita com LAN, VPN, bridges ou outras redes libvirt.

```
ip route
sudo virsh -c qemu:///system net-list --all

if ! sudo virsh -c qemu:///system net-info rhcsa-lab >/dev/null 2>&1; then

cat > /tmp/rhcsa-lab.xml <<'XML'
<network>
  <name>rhcsa-lab</name>
  <forward mode='nat'/>
  <bridge name='virbr-rhcsa' stp='on' delay='0'/>
  <domain name='lab.test'/>
  <ip address='192.168.100.1' netmask='255.255.255.0'>
    <dhcp>
      <range start='192.168.100.100' end='192.168.100.199'/>
    </dhcp>
  </ip>
</network>
XML

  sudo virsh -c qemu:///system net-define /tmp/rhcsa-lab.xml
  sudo virsh -c qemu:///system net-start rhcsa-lab
  sudo virsh -c qemu:///system net-autostart rhcsa-lab
fi

sudo virsh -c qemu:///system net-info rhcsa-lab
sudo virsh -c qemu:///system net-dumpxml rhcsa-lab
ip -br addr show virbr-rhcsa
```

Resultado esperado: bridge virbr-rhcsa com 192.168.100.1/24.

6.6 Validar a imagem-base

A imagem-base e o arquivo SHA-256 devem estar em ~/Downloads.

Download: Baixar imagem-base do laboratório

```
cd ~/Downloads

ls -lh rhel-rhcsa-base-v1.qcow2*
sha256sum -c rhel-rhcsa-base-v1.qcow2.sha256

qemu-img check rhel-rhcsa-base-v1.qcow2
qemu-img info rhel-rhcsa-base-v1.qcow2
```

Não prossiga se o checksum falhar ou se qemu-img check reportar inconsistências.

6.7 Preservar a imagem-mestre

O diretório canônico usado pelo Runbook é /var/lib/libvirt/repo-images.

```
sudo mkdir -p /var/lib/libvirt/repo-images

sudo cp \
  ~/Downloads/rhel-rhcsa-base-v1.qcow2 \
  /var/lib/libvirt/repo-images/

sudo chmod 0444 \
  /var/lib/libvirt/repo-images/rhel-rhcsa-base-v1.qcow2

BASE_IMAGE=/var/lib/libvirt/repo-images/rhel-rhcsa-base-v1.qcow2
IMAGE_DIR=/var/lib/libvirt/images

echo "$BASE_IMAGE"
echo "$IMAGE_DIR"

sudo qemu-img check "$BASE_IMAGE"
```

Não inicialize a imagem-mestre. Ela é apenas a origem para as cópias independentes das três VMs.

7. Criação das máquinas virtuais

O caminho operacional validado pelo Runbook utiliza a imagem-base QCOW2. O método manual por ISO continua disponível na documentação do repositório, mas não é o caminho reproduzido nesta sequência testada.

7.1 Criar os discos das VMs

```
sudo virsh -c qemu:///system list --all

sudo qemu-img convert -p -O qcow2 \
  "$BASE_IMAGE" \
  "$IMAGE_DIR/bastion.qcow2"

sudo qemu-img convert -p -O qcow2 \
  "$BASE_IMAGE" \
  "$IMAGE_DIR/servera.qcow2"

sudo qemu-img convert -p -O qcow2 \
  "$BASE_IMAGE" \
  "$IMAGE_DIR/serverb.qcow2"

sudo qemu-img create -f qcow2 "$IMAGE_DIR/servera-vdb.qcow2" 5G
sudo qemu-img create -f qcow2 "$IMAGE_DIR/servera-vdc.qcow2" 5G
sudo qemu-img create -f qcow2 "$IMAGE_DIR/servera-vdd.qcow2" 3G
sudo qemu-img create -f qcow2 "$IMAGE_DIR/servera-vde.qcow2" 2G

sudo qemu-img create -f qcow2 "$IMAGE_DIR/serverb-vdb.qcow2" 5G
sudo qemu-img create -f qcow2 "$IMAGE_DIR/serverb-vdc.qcow2" 3G

sudo virsh -c qemu:///system pool-refresh default

sudo qemu-img check "$IMAGE_DIR/bastion.qcow2"
sudo qemu-img check "$IMAGE_DIR/servera.qcow2"
sudo qemu-img check "$IMAGE_DIR/serverb.qcow2"
```

Os discos extras devem permanecer sem partições e sem filesystem. /dev/vda é o disco de sistema e deve ser tratado como protegido.

7.2 Criar o bastion

```
sudo virt-install \
  --connect qemu:///system \
  --name bastion \
  --memory 2048 \
  --vcpus 2 \
  --cpu host \
  --osinfo detect=on,require=off \
  --disk path="$IMAGE_DIR/bastion.qcow2",format=qcow2,bus=virtio \
  --network network=rhcsa-lab,model=virtio \
  --import \
  --graphics vnc,listen=127.0.0.1 \
  --noautoconsole
```

7.3 Criar o servera

```
sudo virt-install \
  --connect qemu:///system \
  --name servera \
  --memory 3072 \
  --vcpus 2 \
  --cpu host \
  --osinfo detect=on,require=off \
  --disk path="$IMAGE_DIR/servera.qcow2",format=qcow2,bus=virtio \
  --disk path="$IMAGE_DIR/servera-vdb.qcow2",format=qcow2,bus=virtio \
  --disk path="$IMAGE_DIR/servera-vdc.qcow2",format=qcow2,bus=virtio \
  --disk path="$IMAGE_DIR/servera-vdd.qcow2",format=qcow2,bus=virtio \
  --disk path="$IMAGE_DIR/servera-vde.qcow2",format=qcow2,bus=virtio \
  --network network=rhcsa-lab,model=virtio \
  --import \
```

```
  --graphics vnc,listen=127.0.0.1 \
  --noautoconsole
```

7.4 Criar o serverb

```
sudo virt-install \
  --connect qemu:///system \
  --name serverb \
  --memory 2048 \
  --vcpus 2 \
  --cpu host \
  --osinfo detect=on,require=off \
  --disk path="$IMAGE_DIR/serverb.qcow2",format=qcow2,bus=virtio \
  --disk path="$IMAGE_DIR/serverb-vdb.qcow2",format=qcow2,bus=virtio \
  --disk path="$IMAGE_DIR/serverb-vdc.qcow2",format=qcow2,bus=virtio \
  --network network=rhcsa-lab,model=virtio \
  --import \
  --graphics vnc,listen=127.0.0.1 \
  --noautoconsole
```

7.5 Validar as VMs

```
sudo virsh -c qemu:///system list --all

sudo virsh -c qemu:///system domblklist bastion
sudo virsh -c qemu:///system domblklist servera
sudo virsh -c qemu:///system domblklist serverb

sudo virsh -c qemu:///system domiflist bastion
sudo virsh -c qemu:///system domiflist servera
sudo virsh -c qemu:///system domiflist serverb
```

7.6 Abrir os consoles

No caminho testado, o acesso inicial foi feito pelo console serial do libvirt:

```
sudo virsh console bastion
sudo virsh console servera
sudo virsh console serverb
```

Para sair de um console virsh, utilize Ctrl +].

> *Também é possível usar virt-manager, Cockpit ou virt-viewer. O Runbook usa virsh console por ser rápido e reproduzível.*

8. Configuração inicial das VMs

Execute cada bloco somente na VM indicada. A sequência abaixo reproduz o Runbook testado, incluindo rede estática, /etc/hosts, usuário student, sudo NOPASSWD, SSH, firewalld, SELinux e preparação de pacotes.

8.1 Bastion

```
sudo virsh console bastion
```

```
sudo hostnamectl set-hostname bastion.lab.test

IFACE="$(nmcli -t -f DEVICE,TYPE,STATE device | awk -F: '$2=="ethernet" && $3=="connected"{print $1; exit}')"
LAB_CONNECTION="$(nmcli -g GENERAL.CONNECTION device show "$IFACE")"

echo "IFACE=$IFACE"
echo "LAB_CONNECTION=$LAB_CONNECTION"

sudo nmcli connection modify "$LAB_CONNECTION" \
  ipv4.method manual \
  ipv4.addresses 192.168.100.10/24 \
  ipv4.gateway 192.168.100.1 \
  ipv4.dns 192.168.100.1 \
  ipv4.dns-search lab.test \
  connection.autoconnect yes

sudo nmcli connection up "$LAB_CONNECTION"

sudo sed -i '/# RHCSA-LAB START/,/# RHCSA-LAB END/d' /etc/hosts

cat <<'EOF' | sudo tee -a /etc/hosts
# RHCSA-LAB START
192.168.100.10 bastion.lab.test bastion
192.168.100.11 servera.lab.test servera
192.168.100.12 serverb.lab.test serverb
# RHCSA-LAB END
EOF

id student >/dev/null 2>&1 || sudo useradd -m -G wheel student
sudo usermod -aG wheel student
echo 'student:redhat' | sudo chpasswd

echo 'student ALL=(ALL) NOPASSWD: ALL' | \
  sudo tee /etc/sudoers.d/rhcsa-lab-student >/dev/null

sudo chmod 440 /etc/sudoers.d/rhcsa-lab-student
sudo visudo -cf /etc/sudoers.d/rhcsa-lab-student

sudo systemctl enable --now sshd
sudo systemctl enable --now firewalld

sudo firewall-cmd --permanent --add-service=ssh
sudo firewall-cmd --reload

sudo systemctl is-enabled sshd
sudo systemctl is-active sshd
sudo systemctl is-enabled firewalld
sudo systemctl is-active firewalld

sudo setenforce 1 2>/dev/null || true
sudo sed -ri 's/^[[:space:]]*SELINUX=.*/SELINUX=enforcing/' /etc/selinux/config

hostname -f
ip -4 -br address
ip route
getenforce
systemctl is-active sshd
systemctl is-active firewalld

getent hosts bastion
getent hosts servera
```

```
getent hosts serverb

cat /etc/machine-id
sudo ssh-keygen -A
sudo ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub

sudo dnf repolist

sudo subscription-manager register

sudo subscription-manager repos \
  --enable=rhel-9-for-x86_64-baseos-rpms \
  --enable=rhel-9-for-x86_64-appstream-rpms

sudo dnf clean all
sudo dnf makecache
sudo dnf repolist

sudo dnf update -y
```

Antes de instalar as dependências do bastion, audite a arquitetura dos pacotes:

```
rpm -qa --qf '%{NAME}.%{ARCH}\n' | grep '\.i686$' || echo "OK: nenhum pacote i686 instalado"

sudo subscription-manager identity
sudo subscription-manager status

sudo dnf install -y \
  openssh-server \
  openssh-clients \
  python3 \
  sudo \
  tar \
  gzip \
  bzip2 \
  xz \
  unzip \
  zip \
  git \
  ansible-core \
  python3-pyyaml

python3 --version
git --version
ansible --version
python3 -c 'import yaml; print("PyYAML OK")'

ping -c 2 192.168.100.1
```

A senha student:redhat é uma credencial didática deste laboratório isolado. Não reutilize essa senha em ambientes reais.

8.2 servera

```
sudo virsh console servera

sudo hostnamectl set-hostname servera.lab.test

IFACE="$(nmcli -t -f DEVICE,TYPE,STATE device | awk -F: '$2=="ethernet" && $3=="connected"{print $1; exit}')"
LAB_CONNECTION="$(nmcli -g GENERAL.CONNECTION device show "$IFACE")"

echo "IFACE=$IFACE"
echo "LAB_CONNECTION=$LAB_CONNECTION"

sudo nmcli connection modify "$LAB_CONNECTION" \
  ipv4.method manual \
  ipv4.addresses 192.168.100.11/24 \
  ipv4.gateway 192.168.100.1 \
  ipv4.dns 192.168.100.1 \
  ipv4.dns-search lab.test \
  connection.autoconnect yes

sudo nmcli connection up "$LAB_CONNECTION"

sudo sed -i '/# RHCSA-LAB START/,/# RHCSA-LAB END/d' /etc/hosts
```

```
cat <<'EOF' | sudo tee -a /etc/hosts
# RHCSA-LAB START
192.168.100.10 bastion.lab.test bastion
192.168.100.11 servera.lab.test servera
192.168.100.12 serverb.lab.test serverb
# RHCSA-LAB END
EOF

id student >/dev/null 2>&1 || sudo useradd -m -G wheel student
sudo usermod -aG wheel student
echo 'student:redhat' | sudo chpasswd

echo 'student ALL=(ALL) NOPASSWD: ALL' | \
  sudo tee /etc/sudoers.d/rhcsa-lab-student >/dev/null

sudo chmod 440 /etc/sudoers.d/rhcsa-lab-student
sudo visudo -cf /etc/sudoers.d/rhcsa-lab-student

sudo systemctl enable --now sshd
sudo systemctl enable --now firewalld

sudo firewall-cmd --permanent --add-service=ssh
sudo firewall-cmd --reload

sudo systemctl is-enabled sshd
sudo systemctl is-active sshd
sudo systemctl is-enabled firewalld
sudo systemctl is-active firewalld

sudo setenforce 1 2>/dev/null || true
sudo sed -ri 's/^[[:space:]]*SELINUX=.*/SELINUX=enforcing/' /etc/selinux/config

hostname -f
ip -4 -br address
ip route
getenforce
systemctl is-active sshd
systemctl is-active firewalld

getent hosts bastion
getent hosts servera
getent hosts serverb

cat /etc/machine-id
sudo ssh-keygen -A
sudo ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub

sudo dnf repolist

sudo subscription-manager register

sudo subscription-manager repos \
  --enable=rhel-9-for-x86_64-baseos-rpms \
  --enable=rhel-9-for-x86_64-appstream-rpms

sudo dnf clean all
sudo dnf makecache
sudo dnf repolist

sudo dnf update -y
```

Audite a arquitetura antes do reboot:

```
rpm -qa --qf '%{NAME}.%{ARCH}\n' | grep '\.i686$' || echo "OK: nenhum pacote i686 instalado"

sudo reboot
```

Depois do reboot, volte ao host KVM e execute sudo virsh console servera. Em seguida continue:

```
sudo subscription-manager identity
sudo subscription-manager status

sudo setenforce 1 2>/dev/null || true
sudo sed -ri 's/^[[:space:]]*SELINUX=.*/SELINUX=enforcing/' /etc/selinux/config
```

```
sudo ssh-keygen -A
sudo systemctl restart sshd

sudo dnf repolist

sudo dnf install -y \
  openssh-server \
  openssh-clients \
  python3 \
  sudo

hostname -f
ip -4 -br address
ip route
getenforce
systemctl is-active sshd
systemctl is-active firewalld
cat /etc/machine-id

sudo ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub

getent hosts bastion
getent hosts servera
getent hosts serverb

ping -c 2 192.168.100.1
ping -c 2 bastion
ping -c 2 serverb

lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINTS
```

Esperado em servera: vda 30G, vdb 5G, vdc 5G, vdd 3G e vde 2G.

8.3 serverb

```
sudo virsh console serverb

sudo hostnamectl set-hostname serverb.lab.test

IFACE="$(nmcli -t -f DEVICE,TYPE,STATE device | awk -F: '$2=="ethernet" && $3=="connected"{print $1; exit}')"
LAB_CONNECTION="$(nmcli -g GENERAL.CONNECTION device show "$IFACE")"

echo "IFACE=$IFACE"
echo "LAB_CONNECTION=$LAB_CONNECTION"

sudo nmcli connection modify "$LAB_CONNECTION" \
  ipv4.method manual \
  ipv4.addresses 192.168.100.12/24 \
  ipv4.gateway 192.168.100.1 \
  ipv4.dns 192.168.100.1 \
  ipv4.dns-search lab.test \
  connection.autoconnect yes

sudo nmcli connection up "$LAB_CONNECTION"

sudo sed -i '/# RHCSA-LAB START/,/# RHCSA-LAB END/d' /etc/hosts

cat <<'EOF' | sudo tee -a /etc/hosts
# RHCSA-LAB START
192.168.100.10 bastion.lab.test bastion
192.168.100.11 servera.lab.test servera
192.168.100.12 serverb.lab.test serverb
# RHCSA-LAB END
EOF

id student >/dev/null 2>&1 || sudo useradd -m -G wheel student
sudo usermod -aG wheel student
echo 'student:redhat' | sudo chpasswd

echo 'student ALL=(ALL) NOPASSWD: ALL' | \
  sudo tee /etc/sudoers.d/rhcsa-lab-student >/dev/null

sudo chmod 440 /etc/sudoers.d/rhcsa-lab-student
sudo visudo -cf /etc/sudoers.d/rhcsa-lab-student
```

```
sudo systemctl enable --now sshd
sudo systemctl enable --now firewalld

sudo firewall-cmd --permanent --add-service=ssh
sudo firewall-cmd --reload

sudo systemctl is-enabled sshd
sudo systemctl is-active sshd
sudo systemctl is-enabled firewalld
sudo systemctl is-active firewalld

sudo setenforce 1 2>/dev/null || true
sudo sed -ri 's/^[[:space:]]*SELINUX=.*/SELINUX=enforcing/' /etc/selinux/config

hostname -f
ip -4 -br address
ip route
getenforce
systemctl is-active sshd
systemctl is-active firewalld

getent hosts bastion
getent hosts servera
getent hosts serverb

cat /etc/machine-id
sudo ssh-keygen -A
sudo ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub

sudo dnf repolist

sudo subscription-manager register

sudo subscription-manager repos \
  --enable=rhel-9-for-x86_64-baseos-rpms \
  --enable=rhel-9-for-x86_64-appstream-rpms

sudo dnf clean all
sudo dnf makecache
sudo dnf repolist

sudo dnf update -y
```

Audite a arquitetura antes do reboot:

```
rpm -qa --qf '%{NAME}.%{ARCH}\n' | grep '\.i686$' || echo "OK: nenhum pacote i686 instalado"

sudo reboot
```

Depois do reboot, volte ao host KVM e execute sudo virsh console serverb. Em seguida continue:

```
sudo subscription-manager identity
sudo subscription-manager status

sudo setenforce 1 2>/dev/null || true
sudo sed -ri 's/^[[:space:]]*SELINUX=.*/SELINUX=enforcing/' /etc/selinux/config

sudo ssh-keygen -A
sudo systemctl restart sshd

sudo dnf repolist

sudo dnf install -y \
  openssh-server \
  openssh-clients \
  python3 \
  tar \
  gzip \
  bzip2 \
  xz \
  unzip \
  zip

hostname -f
ip -4 -br address
```

```
ip route
getenforce
systemctl is-active sshd
systemctl is-active firewalld
cat /etc/machine-id

sudo ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub

getent hosts bastion
getent hosts servera
getent hosts serverb

ping -c 2 192.168.100.1
ping -c 2 bastion
ping -c 2 servera

lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINTS
```

Esperado em serverb: vda 30G, vdb 5G e vdc 3G.

8.4 Critério para avançar

- bastion.lab.test usa 192.168.100.10/24;
- servera.lab.test usa 192.168.100.11/24;
- serverb.lab.test usa 192.168.100.12/24;
- gateway 192.168.100.1 e resolução local funcionam;
- student existe e possui sudo NOPASSWD nas três VMs;
- sshd e firewalld estão ativos;
- SELinux está Enforcing;
- machine-id e SSH host keys são individuais;
- os discos extras permanecem intactos;
- não há pacotes i686 introduzidos pela preparação.

9. Preparação do bastion, Git e SSH

9.1 Validar a rede a partir do bastion

```
hostname -f
id -un

getent hosts servera
getent hosts serverb

ping -c 2 servera
ping -c 2 serverb

timeout 5 bash -c '</dev/tcp/servera/22' && echo "servera: SSH OK"
timeout 5 bash -c '</dev/tcp/serverb/22' && echo "serverb: SSH OK"
```

9.2 Criar a chave SSH dedicada

```
mkdir -p ~/.ssh
chmod 700 ~/.ssh

ssh-keygen \
  -t ed25519 \
  -f ~/.ssh/id_ed25519_rhcsa_lab \
  -N '' \
  -C 'rhcsa-lab'

chmod 600 ~/.ssh/id_ed25519_rhcsa_lab
chmod 644 ~/.ssh/id_ed25519_rhcsa_lab.pub

ssh-keygen -lf ~/.ssh/id_ed25519_rhcsa_lab.pub
```

A chave é criada sem passphrase porque o laboratório precisa de automação não interativa. Ela deve permanecer somente no bastion.

9.3 Conferir fingerprints dos hosts

No console de servera:

```
sudo ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub
```

No console de serverb:

```
sudo ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub
```

As fingerprints devem ser diferentes.

9.4 Copiar a chave para os nós

```
ssh-copy-id \
  -i ~/.ssh/id_ed25519_rhcsa_lab.pub \
  student@servera

ssh-copy-id \
  -i ~/.ssh/id_ed25519_rhcsa_lab.pub \
  student@serverb
```

Na primeira conexão, compare a fingerprint exibida com a fingerprint conferida diretamente no console. A senha didática do laboratório é redhat.

9.5 Criar ~/.ssh/config e validar sudo não interativo

```
cat > ~/.ssh/config <<'EOF'
Host servera
    HostName 192.168.100.11
    User student
    IdentityFile ~/.ssh/id_ed25519_rhcsa_lab
    IdentitiesOnly yes

Host serverb
    HostName 192.168.100.12
```

```
    User student
    IdentityFile ~/.ssh/id_ed25519_rhcsa_lab
    IdentitiesOnly yes
EOF

chmod 600 ~/.ssh/config

ssh servera hostname -f
ssh serverb hostname -f

ssh -o BatchMode=yes servera 'sudo -n true' \
  && echo "servera: SSH + sudo OK"

ssh -o BatchMode=yes serverb 'sudo -n true' \
  && echo "serverb: SSH + sudo OK"

sudo -n true \
  && echo "bastion: sudo OK"
```

9.6 Clonar o projeto somente depois de rede, SSH e sudo estarem prontos

```
cd ~

test ! -e ~/rhcsa-ex200-v10-lab

git clone \
  https://github.com/eduardoandradej/rhcsa-ex200-v10-lab.git

cd ~/rhcsa-ex200-v10-lab

git status
git log -1 --oneline --decorate
git describe --tags --always
```

9.7 Instalar o CLI lab

```
cd ~/rhcsa-ex200-v10-lab

bash bootstrap/10-install-lab-cli.sh

source ~/.bashrc

command -v lab
lab --version
```

10. Configuração e validação do Ansible

Este capítulo usa a implementação corrigida do bootstrap de dependências. A baseline dos hosts gerenciados é x86_64/noarch e o processo deve falhar se um RPM i686 aparecer no bundle ou após a instalação.

10.1 Validar Ansible e inventário

```
cd ~/rhcsa-ex200-v10-lab/ansible

ansible --version
ansible-config dump --only-changed

ansible-inventory --graph

ansible control -m ansible.builtin.ping
ansible managed -m ansible.builtin.ping
ansible all -m ansible.builtin.ping
```

Todos os hosts devem retornar SUCCESS/pong.

10.2 Como o bootstrap corrigido funciona

O playbook ansible/playbooks/00-bootstrap-dependencies.yml foi endurecido para evitar dependências multilib inesperadas. O fluxo agora:

- instala no bastion as ferramentas do plano de controle, incluindo dnf-plugins-core e createrepo_c;
- baixa o bundle com dnf download --resolve --alldeps --arch=x86_64,noarch;
- interrompe a execução se encontrar qualquer *.i686.rpm;
- gera metadados de repositório com createrepo_c;
- copia o repositório local para servera e serverb;
- instala os pacotes pelos nomes definidos em managed_baseline_packages, com os repositórios remotos desabilitados;
- desabilita weak dependencies na instalação offline;
- verifica depois da transação que nenhum pacote i686 foi instalado.

```
dnf -y download \
  --resolve \
  --alldeps \
  --arch=x86_64,noarch \
  --destdir /var/tmp/rhcsa-baseline-rpms \
  <managed_baseline_packages>

createrepo_c /var/tmp/rhcsa-baseline-rpms
```

A restrição de arquitetura e a instalação via repositório local evitam que o wildcard de RPMs introduza pacotes i686 na baseline.

10.3 Instalar e validar dependências

```
cd ~/rhcsa-ex200-v10-lab

bash bootstrap/20-install-dependencies.sh
```

Confirme que o bundle no bastion não contém RPM i686:

```
find /var/tmp/rhcsa-baseline-rpms \
  -maxdepth 1 \
  -type f \
  -name '*.i686.rpm' \
  -print
```

Resultado esperado: nenhuma saída.

Confirme também nos hosts gerenciados:

```
cd ~/rhcsa-ex200-v10-lab/ansible
```

```
ansible managed -b -m ansible.builtin.shell \
  -a "rpm -qa --qf '%{NAME}.%{ARCH}\n' | grep '\.i686$' || true"
```

Resultado esperado: SUCCESS em servera e serverb, com stdout vazio.

10.4 Executar as validações explícitas

```
cd ~/rhcsa-ex200-v10-lab/ansible

ansible all -m ansible.builtin.ping

ansible-playbook playbooks/validate.yml

ansible-playbook playbooks/00-validate-dependencies.yml
```

10.5 Validar o CLI e o catálogo

```
cd ~/rhcsa-ex200-v10-lab

lab status

lab list

lab list --objective obj01

lab show obj01-01
```

Neste ponto, o plano de controle deve estar funcional e a baseline pronta para ser congelada em snapshot.

11. Baseline e snapshots

Execute este capítulo no host KVM, não no bastion. O objetivo é preservar um estado limpo de servera e serverb antes dos estudos.

11.1 Verificar discos e snapshots existentes

```
sudo virsh domblklist servera
sudo virsh domblklist serverb

sudo virsh snapshot-list servera
sudo virsh snapshot-list serverb
```

11.2 Desligar os nós gerenciados

```
sudo virsh shutdown servera
sudo virsh shutdown serverb

watch -n 2 'sudo virsh list --all'
```

Saia do watch com Ctrl+C quando ambos estiverem shut off. Depois confirme:

```
sudo virsh domstate servera
sudo virsh domstate serverb
```

Só prossiga se ambos estiverem desligados e se os formatos dos discos suportarem o tipo de snapshot pretendido.

11.3 Criar baseline-pre-labs

```
sudo virsh snapshot-create-as \
  servera \
  baseline-pre-labs \
  --description "Baseline RHCSA limpo antes dos laboratórios"

sudo virsh snapshot-create-as \
  serverb \
  baseline-pre-labs \
  --description "Baseline RHCSA limpo antes dos laboratórios"
```

11.4 Validar snapshots e reiniciar

```
sudo virsh snapshot-info \
  servera \
  baseline-pre-labs

sudo virsh snapshot-info \
  serverb \
  baseline-pre-labs

sudo virsh snapshot-list servera
sudo virsh snapshot-list serverb

sudo virsh start servera
sudo virsh start serverb

sudo virsh list
```

Snapshot não substitui backup. Use-o como mecanismo de recuperação rápida do ambiente de estudo.

12. Primeiro exercício

12.1 Check final do ambiente

Volte ao bastion como student:

```
ssh -o BatchMode=yes \
  -o ConnectTimeout=5 \
  servera \
  hostname -f

ssh -o BatchMode=yes \
  -o ConnectTimeout=5 \
  serverb \
  hostname -f

cd ~/rhcsa-ex200-v10-lab/ansible

ansible all -m ansible.builtin.ping

cd ~/rhcsa-ex200-v10-lab

lab status
lab list --objective obj01
lab show obj01-01
```

12.2 Começar a estudar

```
cd ~/rhcsa-ex200-v10-lab

lab start obj01-01
```

Leia a tarefa e resolva manualmente. Depois avalie:

```
lab grade
```

Ao concluir:

```
lab finish
```

Consulte o próximo exercício:

```
lab show obj01-02
```

12.3 Checklist final

- KVM/libvirt e Cockpit funcionam no host;
- pool default está ativo;
- rede rhcsa-lab está ativa e virbr-rhcsa usa 192.168.100.1/24;
- bastion, servera e serverb existem;
- snapshot baseline-pre-labs existe em servera e serverb;
- bastion usa 192.168.100.10, Git, Ansible, Python/PyYAML e CLI lab;
- servera usa 192.168.100.11 e possui vdb 5G, vdc 5G, vdd 3G, vde 2G;
- serverb usa 192.168.100.12 e possui vdb 5G, vdc 3G;
- SELinux está Enforcing e firewalld/sshd estão ativos;
- SSH por chave e sudo NOPASSWD funcionam;
- ansible all -m ansible.builtin.ping funciona;
- bootstrap/20-install-dependencies.sh concluiu sem i686;
- lab status, lab list e lab show obj01-01 funcionam.

12.4 Atalho operacional para os próximos dias

No host KVM:

```
sudo virsh start bastion 2>/dev/null || true
sudo virsh start servera 2>/dev/null || true
```

```
sudo virsh start serverb 2>/dev/null || true
sudo virsh list
```

No bastion:

```
cd ~/rhcsa-ex200-v10-lab
git pull --ff-only

lab status
lab list
lab show obj01-01
```

Para começar:

```
lab start obj01-01
```

12.5 Regras operacionais

- o host KVM é infraestrutura fora de banda; não entregue controle de snapshots/libvirt ao bastion;
- bastion é o Control Plane; não use-o para exercícios destrutivos;
- servera e serverb são os nós de prática;
- /dev/vda é o disco de sistema protegido;
- os discos extras existem para os exercícios de storage;
- não desabilite SELinux ou firewalld para contornar falhas;
- não use StrictHostKeyChecking=no como correção permanente;
- a chave SSH privada do laboratório permanece somente no bastion;
- não versione senhas, chaves privadas, tokens ou credenciais RHSM.

13. CLI lab e fluxo operacional

O comando lab é a interface operacional do ambiente. Ele descobre o catálogo, controla o estado do exercício ativo, aciona playbooks de preparação/limpeza e executa graders. O fluxo normal mantém apenas um laboratório ativo por vez.

Comando	Finalidade
lab status	Verifica nós, ferramentas, plataforma e estado do laboratório ativo.
lab list	Lista o catálogo e permite filtrar por objetivo.
lab show <id>	Mostra metadados e enunciado do laboratório.
lab start <id>	Prepara o cenário e registra o exercício como ativo.
lab grade <id>	Executa o grader e apresenta a pontuação.
lab finish <id>	Executa o cleanup e encerra o estado ativo.

```
lab status
lab list
lab show obj01-01
lab start obj01-01
ssh student@servera
lab grade obj01-01
lab finish obj01-01
```

13.1 Fluxo de uma sessão

- Confirme Environment: READY com lab status.
- Escolha um item do catálogo com lab list ou lab list --objective ....
- Leia o enunciado com lab show antes de iniciar.
- Execute lab start e resolva manualmente no host alvo.
- Use documentação local do sistema antes de consultar soluções.
- Execute lab grade; corrija o estado se necessário e rode o grader novamente.
- Somente ao terminar, execute lab finish e confirme a limpeza.

14. Estratégia de estudo e avaliação por estado final

Os graders foram desenhados para observar o estado real do sistema. Em vez de exigir uma sequência textual de comandos, podem verificar existência, conteúdo, permissões, ownership, serviços, configuração persistente, montagem, política SELinux, regras de firewall e outros efeitos observáveis.

> ***Observação:*** *As soluções de referência em tests/reference-solutions/ existem para QA da engenharia do laboratório. O fluxo normal do estudante é resolver manualmente antes de consultar qualquer solução.*

14.1 Rotina recomendada

- Faça os objetivos em ordem até construir fluência operacional.
- Refaça labs com pontuação inferior a 100% após um intervalo, sem consultar solução.
- Nos desafios integrados, trabalhe a partir do estado final solicitado e documente mentalmente a evidência de cada requisito.
- Repita os dois drills manuais de boot/recovery em console e, quando possível, também em um ambiente RHEL 10.
- Use snapshots como recuperação out-of-band, não como substituto de `lab finish`.

PARTE IV

Catálogo completo de laboratórios

RHCSA EX200 v10 Lab

15. Catálogo completo dos 100 itens

O catálogo abaixo foi extraído dos arquivos lab.yml presentes no repositório desta edição. Ele registra ID, título, alvo, dificuldade, duração, compatibilidade e status operacional.

Objective 01 — Essential Tools

ID	Laboratório	Alvo	Dif.	Tempo	Compatibilidade	Estado
obj01-01	Operações com arquivos	servera	1	10 min	exact	ready
obj01-02	Redirecionamento e pipes	servera	2	12 min	exact	ready
obj01-03	grep e expressões regulares	servera	2	12 min	exact	ready
obj01-04	SSH e troca de usuários	servera, serverb	2	12 min	exact	ready
obj01-05	Arquivos e compressão	servera	2	12 min	exact	ready
obj01-06	Edição de arquivos de texto	servera	2	10 min	exact	ready
obj01-07	Hard links e symbolic links	servera	2	10 min	exact	ready
obj01-08	Permissões de arquivos	servera	2	12 min	exact	ready
obj01-09	Documentação do sistema	servera	2	10 min	exact	ready
obj01-10	Desafio integrado Essential Tools	servera, serverb	3	25 min	exact	ready

Objective 02 — Users & Groups

ID	Laboratório	Alvo	Dif.	Tempo	Compatibilidade	Estado
obj02-01	Identidade e bases locais	servera	1	10 min	exact	ready
obj02-02	Criar, modificar e remover usuários	servera	2	14 min	exact	ready
obj02-03	Criar e modificar grupos	servera	2	12 min	exact	ready
obj02-04	Grupos primários e suplementares	servera	2	14 min	exact	ready
obj02-05	Senhas e password aging	servera	2	15 min	exact	ready
obj02-06	Sudo e sudoers.d	servera	3	15 min	exact	ready
obj02-07	Lock, expiração e nologin	servera	2	14 min	exact	ready
obj02-08	Desafio integrado Users & Groups	serverb	3	25 min	exact	ready

Objective 03 — RPM, DNF & Repositories

ID	Laboratório	Alvo	Dif.	Tempo	Compatibilidade	Estado
obj03-01	Consultar a base RPM instalada	servera	1	12 min	exact	ready
obj03-02	Inspecionar e extrair um arquivo RPM	servera	2	18 min	exact	ready
obj03-03	Descobrir software com DNF	servera	2	15 min	exact	ready
obj03-04	Instalar e remover pacotes com DNF	servera	2	15 min	exact	ready
obj03-05	Configurar um repositório DNF	servera	2	15 min	exact	ready
obj03-06	Atualizar pacote por repositório de errata	servera	3	18 min	exact	ready
obj03-07	Auditar transações com DNF history	servera	2	18 min	exact	ready
obj03-08	Desafio integrado de gerenciamento de software	serverb	3	28 min	exact	ready

Objective 04 — Simple Shell Scripts

ID	Laboratório	Alvo	Dif.	Tempo	Compatibilidade	Estado
obj04-01	Parâmetros posicionais e validação de entrada	servera	1	15 min	exact	ready
obj04-02	Condições com test e colchetes	servera	2	16 min	exact	ready
obj04-03	Decisões a partir do status de comandos	servera	2	16 min	exact	ready
obj04-04	Processar saída de comandos em um script	servera	2	18 min	exact	ready
obj04-05	Loop sobre argumentos da linha de comando	servera	2	16 min	exact	ready
obj04-06	Loop sobre entrada de arquivo	servera	2	18 min	exact	ready
obj04-07	Relatório de usuários com loop e saída de comandos	servera	3	22 min	exact	ready
obj04-08	Desafio integrado de shell scripting	serverb	3	30 min	exact	ready

Objective 05 — Processes, Scheduling, Tuning & systemd

ID	Laboratório	Alvo	Dif.	Tempo	Compatibilidade	Estado
obj05-01	Identificar processos, PIDs e estados	servera	1	15 min	exact	ready
obj05-02	Controlar jobs em foreground e background	servera	2	20 min	exact	ready

ID	Laboratório	Alvo	Dif.	Tempo	Compatibilidade	Estado
obj05-03	Controlar processos com sinais	servera	2	18 min	exact	ready
obj05-04	Identificar e encerrar processos intensivos	servera	3	22 min	exact	ready
obj05-05	Ajustar prioridade com nice e renice	servera	2	20 min	exact	ready
obj05-06	Gerenciar perfis de ajuste com TuneD	servera	2	18 min	exact	ready
obj05-07	Identificar units e estados do systemd	servera	2	18 min	exact	ready
obj05-08	Start, restart e reload de serviço	servera	3	22 min	exact	ready
obj05-09	Enable, disable, mask e dependências	servera	3	22 min	exact	ready
obj05-10	Desafio integrado de processos e systemd	serverb	4	35 min	exact	ready

Objective 06 — NetworkManager, IP Networking & Name Resolution

ID	Laboratório	Alvo	Dif.	Tempo	Compatibilidade	Estado
obj06-01	Inventariar dispositivos e conexões de rede	servera	1	15 min	concept-and-state	ready
obj06-02	Criar perfil IPv4 estático com nmcli	servera	2	20 min	concept-and-state	ready
obj06-03	Modificar perfil, gateway e IPv4 secundário	servera	2	22 min	concept-and-state	ready
obj06-04	Configurar perfil dual-stack IPv4 e IPv6	servera	3	25 min	concept-and-state	ready
obj06-05	Editar perfil NetworkManager em formato keyfile	servera	3	25 min	concept-and-state	ready
obj06-06	Configurar hostname e resolução local	servera	2	22 min	concept-and-state	ready
obj06-07	Configurar DNS e search domain em perfil NetworkManager	servera	2	20 min	concept-and-state	ready
obj06-08	Desafio integrado de NetworkManager e resolução	serverb	4	35 min	concept-and-state	ready

Objective 07 — Scheduling, Temporary Files, Logging & Time

ID	Laboratório	Alvo	Dif.	Tempo	Compatibilidade	Estado
obj07-01	Gerenciar tarefa recorrente com systemd timer	servera	2	22 min	concept-and-state	ready
obj07-02	Gerenciar arquivos temporários com systemd-tmpfiles	servera	2	20 min	concept-and-state	ready
obj07-03	Agendar tarefa recorrente em /etc/cron.d	servera	2	18 min	concept-and-state	ready
obj07-04	Roteamento de eventos com rsyslog	servera	2	20 min	concept-and-state	ready

ID	Laboratório	Alvo	Dif.	Tempo	Compatibilidade	Estado
obj07-05	Filtrar eventos com journalctl	servera	2	20 min	concept-and-state	ready
obj07-06	Configurar journal persistente	servera	3	24 min	concept-and-state	ready
obj07-07	Configurar fuso horário e verificar Chrony	servera	2	20 min	concept-and-state	ready
obj07-08	Desafio integrado de scheduling e logs	serverb	4	35 min	concept-and-state	ready

Objective 08 — Partitions, Filesystems, Swap & LVM

ID	Laboratório	Alvo	Dif.	Tempo	Compatibilidade	Estado
obj08-01	Inventariar dispositivos de bloco	servera	1	15 min	concept-and-state	ready
obj08-02	Criar GPT e partição	servera	2	18 min	concept-and-state	ready
obj08-03	Criar XFS e montar temporariamente	servera	2	18 min	concept-and-state	ready
obj08-04	Montar XFS persistentemente por UUID	servera	3	22 min	concept-and-state	ready
obj08-05	Criar swap persistente em partição	servera	3	20 min	concept-and-state	ready
obj08-06	Desafio integrado de Basic Storage	servera	4	35 min	concept-and-state	ready
obj08-07	Criar PV, VG e LV	servera	3	28 min	concept-and-state	ready
obj08-08	Criar XFS em LV e persistir mount	servera	3	25 min	concept-and-state	ready
obj08-09	Estender LV e filesystem XFS	servera	3	24 min	concept-and-state	ready
obj08-10	Adicionar PV e estender VG	servera	3	28 min	concept-and-state	ready
obj08-11	Criar swap em LVM	servera	3	28 min	concept-and-state	ready
obj08-12	Desafio integrado de Storage e LVM	servera	5	45 min	concept-and-state	ready

Objective 09 — Boot, GRUB & Recovery

ID	Laboratório	Alvo	Dif.	Tempo	Compatibilidade	Estado
obj09-01	Inventário de boot, kernel e GRUB	servera	2	18 min	concept-and-state	ready
obj09-02	Argumentos persistentes do kernel com grubby	servera	3	22 min	exact	ready
obj09-03	Gerenciar o target padrão do systemd	servera	3	18 min	exact	ready
obj09-04	Rescue e emergency pelo console	servera	4	30 min	manual-console-rhel10-aligned	manual-only

ID	Laboratório	Alvo	Dif.	Tempo	Compatibilidade	Estado
obj09-05	Diagnosticar e corrigir falha de fstab	servera	4	28 min	concept-and-state	ready
obj09-06	Recuperação de acesso root pelo boot loader	servera	5	40 min	manual-console-rhel10-aligned	manual-only
obj09-07	Desafio integrado de boot e recuperação	servera	4	35 min	concept-and-state	ready

Objective 10 — SELinux & firewalld

ID	Laboratório	Alvo	Dif.	Tempo	Compatibilidade	Estado
obj10-01	Inventário de SELinux e contextos	servera	2	18 min	concept-and-state	ready
obj10-02	Restaurar SELinux enforcing atual e persistente	servera	3	20 min	exact	ready
obj10-03	Contexto SELinux persistente com semanage e restorecon	servera	3	24 min	exact	ready
obj10-04	Boolean SELinux persistente	servera	3	22 min	exact	ready
obj10-05	Diagnóstico de AVC e correção de contexto	servera	4	32 min	concept-and-state	ready
obj10-06	Zona, origem, serviço e porta no firewalld	servera	3	25 min	exact	ready
obj10-07	Rotular porta não padrão para httpd	servera	4	28 min	exact	ready
obj10-08	Persistência de porta na zona ativa	servera	3	22 min	exact	ready
obj10-09	Desafio integrado SELinux e firewalld	servera	5	40 min	concept-and-state	ready

Objective 11 — NFS & AutoFS

ID	Laboratório	Alvo	Dif.	Tempo	Compatibilidade	Estado
obj11-01	Descobrir, montar e desmontar NFS	servera	2	20 min	concept-and-state	ready
obj11-02	Montagem NFS persistente no fstab	servera	3	24 min	concept-and-state	ready
obj11-03	Mapa direto do AutoFS	servera	3	28 min	concept-and-state	ready
obj11-04	Mapa indireto com wildcard no AutoFS	servera	4	30 min	concept-and-state	ready
obj11-05	Automount NFS via systemd e fstab	servera	4	30 min	concept-and-state	ready
obj11-06	Desafio integrado NFS e AutoFS	servera	5	40 min	concept-and-state	ready

Objective 12 — Comprehensive EX200 Challenges

ID	Laboratório	Alvo	Dif.	Tempo	Compatibilidade	Estado
obj12-01	Desafio final de usuários, permissões, ACL e shell	servera	5	35 min	concept-and-state	ready
obj12-02	Desafio final de serviços, agendamento e logs	servera	5	35 min	concept-and-state	ready
obj12-03	Desafio final de partição, LVM, XFS e swap	servera	5	45 min	concept-and-state	ready
obj12-04	Desafio final de boot e manutenção persistente	servera	5	35 min	concept-and-state	ready
obj12-05	Desafio final de SELinux, firewalld e Apache	servera	5	40 min	concept-and-state	ready
obj12-06	Simulado final integrado	servera	5	45 min	concept-and-state	ready

15.1 Totais operacionais

Métrica	Quantidade
Itens de aprendizagem	100
Labs ready	98
Drills manual-only	2
Reset policy ansible	85
Reset policy guarded-storage-reset	13
Reset policy manual-console	2
Compatibilidade exact	52
Compatibilidade concept-and-state	46
Compatibilidade manual-console-rhel10-aligned	2

16. Boot, recuperação manual e segurança de armazenamento

16.1 Dois exercícios deliberadamente manuais

O Objective 09 possui sete itens. Cinco são automatizados e dois são manual-only: obj09-04 (rescue/emergency pelo console) e obj09-06 (recuperação de acesso root pelo boot loader). A escolha evita conceder ao bastion controle privilegiado out-of-band do hipervisor apenas para automatizar interação com GRUB.

Os drills devem ser praticados pelo console gráfico/libvirt. O repositório também registra a diferença de contexto entre o runtime local RHEL 9.8 e o alvo RHEL 10 para recuperação de boot.

16.2 Contrato de segurança do Objective 08

O Objective 08 é intencionalmente destrutivo apenas em discos de scratch. Os dispositivos do sistema nunca são alvos autorizados.

Categoria	**Dispositivos / objetos**
Protegidos	/dev/vda, /dev/vda1, /dev/vda2, rhel_servera/root, rhel_servera/swap
Scratch autorizado em servera	/dev/vdb (5 GiB), /dev/vdc (5 GiB), /dev/vdd (3 GiB), /dev/vde (2 GiB)

- O preflight valida host, tipo de dispositivo e faixa de tamanho antes de um lab destrutivo.
- O reset normal é limitado pela allowlist de scratch disks.
- Labs destrutivos salvam /etc/fstab antes do trabalho e restauram a cópia no finish.
- Snapshots permanecem como recuperação out-of-band para falhas anormais.

PARTE V

Engenharia, QA e manutenção

RHCSA EX200 v10 Lab

17. Estrutura do repositório e contrato de um lab

```
rhcsa-ex200-v10-lab/
├── ansible/
│   ├── ansible.cfg
│   ├── inventory/
│   ├── playbooks/
│   └── vars/
├── bin/lab
├── bootstrap/
├── docs/
│   ├── setup/
│   ├── releases/
│   └── development/
├── labctl/
├── labs/
├── tests/
│   ├── reference-solutions/
│   ├── integration-reference.sh
│   └── release-gate.sh
├── README.md
├── SUPPORT.md
└── LICENSE
```

17.1 Contrato de um laboratório ready

Cada laboratório do catálogo possui um manifesto `lab.yml` e, quando automatizado, os artefatos necessários para preparar, avaliar e limpar o cenário. O catálogo é descoberto pelo CLI a partir desses manifests.

Artefato	Função
lab.yml	Metadados: ID, título, objetivo, alvo, dificuldade, duração, compatibilidade, reset policy e status.
prompt.pt.md / prompt.en.md	Enunciado do estudante.
setup.yml	Preparação automatizada do cenário.
grade.py	Validação do estado final.
finish.yml	Cleanup e restauração do cenário.
solution.md	Referência de solução; não faz parte do fluxo normal do estudante.

17.2 Separação de responsabilidades

Camada	Responsabilidade
Ansible	Preparação, dependências, estado inicial e cleanup.
Python / labctl	Catálogo, orquestração, estado ativo, CLI e graders.
Estudante	Resolução manual do cenário no sistema alvo.
Host KVM	Controle out-of-band, snapshots e recuperação.

18. Release gate, integração de referência e manutenção

18.1 Gate estático

```
bash tests/release-gate.sh
```

O gate valida sintaxe Python, metadados YAML, contrato dos labs `ready`, sintaxe Ansible, smoke test do CLI e verificações específicas de segurança/portabilidade implementadas no conjunto de testes do repositório.

18.2 Integração de referência

```
bash tests/integration-reference.sh
```

O runner de referência executa o ciclo `lab start` → solução de referência → `lab grade` exigindo 100% → `lab finish` para os laboratórios automatizados selecionados. A existência desse runner não transforma a solução automatizada em método de estudo; sua finalidade é QA.

```
bash tests/integration-reference.sh obj01-04 obj01-05 obj01-06 obj01-07
```

18.3 Checklist de release

- Executar `bash tests/release-gate.sh`.
- Executar a integração de referência pertinente à alteração.
- Confirmar que o fluxo manual do estudante permanece separado das soluções de QA.
- Revisar segurança de storage, rede, SSH, SELinux e cleanup quando a mudança tocar essas áreas.
- Atualizar documentação e release notes quando o comportamento público mudar.

18.4 Fluxo Git recomendado para manutenção documental

```
git switch -c docs/minha-alteracao
# editar e validar
git diff --check
git add <arquivos>
git diff --cached --check
git commit -m "docs: describe change"
git push -u origin docs/minha-alteracao
```

Para alterações maiores, o fluxo por branch e Pull Request facilita revisão antes do merge em `main`. Em correções pequenas e controladas, o mantenedor pode adotar fluxo mais simples conforme sua política de contribuição.

19. Releases e roadmap

Versão	Nota de release
v1.2.0	v1.2.0 — Objective 06 Networking
v1.2.1	v1.2.1 — OBJ06 routing contract hotfix
v1.2.2	v1.2.2 — OBJ06 IPv6 grader hotfix
v1.2.3	v1.2.3 — OBJ06 keyfile grader permission hotfix
v1.3.0	v1.3.0 — Objective 07 Scheduling, Logging & Time
v1.3.1	v1.3.1 — OBJ07 tmpfiles grader permission hotfix
v1.3.2	v1.3.2 — OBJ07 tmpfiles age-semantics hotfix
v1.4.0	v1.4.0 — Objective 08 Storage
v1.4.1	v1.4.1 — OBJ08 safety-bootstrap hotfix
v1.4.2	v1.4.2 — OBJ08 swap identity hotfix
v1.4.3	v1.4.3 — OBJ08 protected-disk release-gate hotfix
v1.4.4	v1.4.4 — OBJ08 bounded LVM setup hotfix
v1.4.5	v1.4.5 — OBJ08 bounded reset and recovery hotfix
v1.4.6	v1.4.6 — interrupted LVM recovery hotfix
v1.4.7	v1.4.7 — timeout containment hotfix
v1.4.8	v1.4.8 — stale LVM signature hotfix
v1.4.9	v1.4.9 — release-gate grep matcher hotfix
v1.4.10	v1.4.10 — LVM swap cleanup hotfix
v1.4.12	v1.4.12 — Objective 03 local repository permission hotfix
v1.4.13	v1.4.13 — Objective 05 non-interactive pause hotfix
v1.5.0	v1.5.0 — Objective 09 Boot, GRUB & Recovery
v1.5.1	v1.5.1 — OBJ09 reference-solver stdin hotfix
v1.5.2	v1.5.2 — OBJ08-02 GPT partition grader portability hotfix
v1.6.0	v1.6.0 — Objective 10 SELinux & firewalld
v1.6.1	v1.6.1 — OBJ10-05 bounded AVC analysis hotfix
v1.7.0	v1.7.0 — Objective 11 NFS & AutoFS
v1.8.0	v1.8.0 — Objective 12 Comprehensive EX200 Challenges
v1.8.1	v1.8.1 — OBJ12-02 grader privileged-log hotfix
v1.8.2	v1.8.2 — OBJ12-03 non-interactive XFS growth hotfix
v1.8.3	v1.8.3 — OBJ12-03 evidence grader hotfix
v1.8.4	v1.8.4 — OBJ12-06 protected report evidence hotfix

19.1 Estado do roadmap

O currículo está concluído em 12 objetivos, com 98 labs automatizados e 2 drills manuais. O roadmap do repositório registra como gate de engenharia a regressão sequencial de referência dos 98 labs automatizados após a validação do Objective 12.

> ***Observação:*** *A presença do runner não deve ser interpretada como evidência automática de que uma regressão completa foi executada após toda alteração documental ou funcional. O histórico de execução deve ser tratado separadamente do código que implementa o runner.*

PARTE VI

Referência e publicação

RHCSA EX200 v10 Lab

20. Referência rápida

20.1 Comandos do estudante

```
lab status
lab list
lab list --objective 08
lab show obj08-01
lab start obj08-01
ssh student@servera
lab grade obj08-01
lab finish obj08-01
```

20.2 Comandos de diagnóstico do ambiente

```
virsh list --all
virsh net-list --all
virsh pool-list --all
ansible all -m ansible.builtin.ping
ansible all -b -m ansible.builtin.command -a 'id'
lsblk
findmnt --verify
getenforce
firewall-cmd --get-active-zones
systemctl --failed
```

20.3 Valores esperados de rede

Elemento	Valor
Rede	rhcsa-lab
CIDR	192.168.100.0/24
Gateway	192.168.100.1
bastion	192.168.100.10
servera	192.168.100.11
serverb	192.168.100.12
Domínio	lab.test

20.4 Downloads e repositório

Repositório do projeto

Baixar imagem-base do laboratório

> ***Observação:*** *A distribuição pública da imagem-base deve observar os termos aplicáveis ao Red Hat Enterprise Linux, conforme registrado na documentação do projeto.*

21. Autoria, apoio, licença e marcas

21.1 Autor e perfis profissionais

Eduardo Andrade

LinkedIn — linkedin.com/in/eduardoandradej

Credly — credly.com/users/eduardoandradej

21.2 Apoie o projeto

O RHCSA EX200 v10 Lab é desenvolvido e mantido de forma independente e disponibilizado gratuitamente. O apoio financeiro é inteiramente opcional e não condiciona acesso ao código, laboratórios, documentação, funcionalidades ou licença.

PIX	**Dados**
Chave	5a41a19c-a773-4fc5-bd8d-821ebbe37c23
Favorecido	Eduardo Andrade

21.3 Independência e marcas

- O projeto não é afiliado à Red Hat.
- O projeto não é patrocinado pela Red Hat.
- O projeto não comercializa conteúdo oficial da Red Hat.
- O projeto não distribui questões reais de certificação.
- O projeto não substitui treinamento oficial, documentação oficial ou Red Hat Learning Subscription.

Red Hat, Red Hat Enterprise Linux, RHEL, RHCSA e demais marcas mencionadas pertencem aos seus respectivos titulares.

21.4 Licença

Código e documentação do repositório são disponibilizados sob a Apache License 2.0. O Apêndice A reproduz fielmente o conteúdo do arquivo `LICENSE` incluído no repositório, que também informa o endereço oficial para consulta dos termos completos.

Copyright 2026 Eduardo Andrade

Apêndice A — Arquivo LICENSE (Apache License 2.0)

Apache License
Version 2.0, January 2004
http://www.apache.org/licenses/

Copyright 2026 Eduardo Andrade

Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at

http://www.apache.org/licenses/LICENSE-2.0

Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.

RHCSA EX200 v10 Lab

Automação para preparar e avaliar.
Administração Linux para ser praticada de verdade.